

API Design & React Query Implementation

This document details the third-party API integration (GNews), internal API routes, React Query setup, and data structures used in the DailyScope News application.

Third-Party API: GNews v4

The application integrates with **GNews API v4** (<https://gnews.io/api/v4/>) for fetching news articles. All GNews API calls are abstracted through the `fetchNews()` service function in `src/services/newsService.ts` .

Endpoint 1: Top Headlines

Base URL: `https://gnews.io/api/v4/top-headlines`

Method: `GET`

Request Parameters (Query String):

Parameter	Type	Required	Description
lang	string	Yes	Language code (always set to "en")
country	string	Yes	Country code (always set to "us")
max	string	Yes	Maximum number of articles (always set to "10")
apikey	string	Yes	GNews API key from <code>process.env.GNEWS_API_KEY</code>
topic	string	Optional	GNews topic filter (e.g., world, nation, business, technology, entertainment, sports, science, health)

When This Endpoint Is Used:

- When `breaking === true` in the `fetchNews()` options, **or**
- When `topic` is provided and no `q` (search query) is given

Response Body:

```
interface NewsApiResponse {
  totalArticles?: number;
  articles: NewsArticle[];
  error?: string; // Added by our code for local validation cases
}

interface NewsArticle {
  title: string;
  description: string;
  content?: string;
  url: string;
  image?: string;
  publishedAt?: string;
  source: NewsSource | string;
}

interface NewsSource {
  id?: string;
  name: string;
  url?: string;
```

```
country?: string;
}
```

Status Codes:

Code	Description	Behavior in Application
200 OK	Successful response	Data is parsed and returned as <code>NewsApiResponse</code>
4xx / 5xx	API errors (rate limit, invalid key, server error, etc.)	Error is logged; application falls back to mock data (returns <code>NewsApiResponse</code> with 10 sample articles)

Error Handling:

- Network errors, timeouts (15-second timeout), or non-OK responses trigger a fallback to mock data
- The application does not throw errors to the caller; it always returns a valid `NewsApiResponse` structure

Endpoint 2: Search

Base URL: `https://gnews.io/api/v4/search`

Method: `GET`

Request Parameters (Query String):

Parameter	Type	Required	Description
q	string	Conditional*	Search query string
lang	string	Yes	Language code (always set to "en")
country	string	Yes	Country code (always set to "us")
max	string	Yes	Maximum number of articles (always set to "10")
apikey	string	Yes	GNews API key from <code>process.env.GNEWS_API_KEY</code>
topic	string	Conditional*	GNews topic filter
page	string	Optional	Page number (available in options but not currently forwarded)
fromDate	string	Optional	Date filter (available in options but not currently forwarded)

* **Conditional Requirements:** At least one of the following must be provided:

- `q` (search query), **or**
- `topic` (topic filter), **or**
- `breaking === true` (which switches to top-headlines endpoint)

When This Endpoint Is Used:

- When a search query (`q`) is provided and `breaking !== true`
- When neither `breaking` nor `topic` is set, but `q` is provided

Response Body:

Same structure as Top Headlines endpoint:

```
interface NewsApiResponse {
  totalArticles?: number;
  articles: NewsArticle[];
}
```

```
error?: string;
}
```

Status Codes:

Code	Description	Behavior in Application
200 OK	Successful response	Data is parsed and returned as NewsApiResponse
Validation Error	Missing query/topic (handled locally)	Returns { error: "Missing query or topic", articles: [] } without calling GNews API
4xx / 5xx	API errors	Error is logged; application falls back to mock data

Error Handling:

- If `q`, `topic`, and `breaking` are all missing, the service returns a validation error object without making an HTTP request
- Network errors, timeouts, or API errors trigger fallback to mock data

Internal API: Next.js Route Handler

Endpoint: `/api/news`

File: `src/app/api/news/route.ts`

Method: `GET`

Request Parameters (Query String):

Parameter	Type	Required	Description
<code>q</code>	string	Optional	Search query string
<code>topic</code>	string	Optional	GNews topic filter
<code>breaking</code>	string	Optional	Set to "1" for breaking news / top headlines
<code>page</code>	string	Optional	Page number
<code>fromDate</code>	string	Optional	Date filter (ISO string)

Example Requests:

```
GET /api/news?q=technology&breaking=0
GET /api/news?topic=business&breaking=1
GET /api/news?breaking=1
```

Response Body:

Success Response (200 OK):

```
{
  "totalArticles": 10,
  "articles": [
    {
      "title": "Article Title",
      "description": "Article description",
      "content": "Full article content...",
    }
  ]
}
```

```
    "url": "https://example.com/article",
    "image": "https://example.com/image.jpg",
    "publishedAt": "2024-01-01T00:00:00Z",
    "source": {
      "name": "Source Name",
      "url": "https://source.com"
    }
  }
]
```

Error Response (400 Bad Request):

```
{
  "error": "Missing query or topic"
}
```

Status Codes:

Code	Description	Response Body
200 OK	Successful fetch (from GNews or mock data)	NewsApiResponse object
400 Bad Request	Validation error (missing required parameters)	{ "error": string }
200 OK (fallback)	Server error caught	Mock news data (NewsApiResponse with 10 sample articles)

Error Handling:

- If `fetchNews()` returns a validation error (`error` field with empty `articles` array), the route returns `400 Bad Request`
- If any other error occurs (network, timeout, exception), the route catches it, logs the error, and returns mock data with `200 OK` status
- This ensures clients always receive a valid response structure

React Query Implementation

Provider Setup

File: `src/app/providers.tsx`

Implementation:

```
'use client';

import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';
import { PropsWithChildren, useState } from 'react';

export default function Providers({ children }: PropsWithChildren) {
  const [queryClient] = useState(() => new QueryClient());
  return (
    <QueryClientProvider client={queryClient}>
      {children}
      <ReactQueryDevtools initialIsOpen={false} />
    </QueryClientProvider>
  );
}
```

```
);  
}
```

Key Points:

- **Single QueryClient Instance:** Created once per app using `useState` to ensure a stable reference
- **Provider Wrapper:** `QueryClientProvider` wraps the entire application, enabling React Query hooks in all client components
- **Devtools:** `ReactQueryDevtools` is included for development debugging (closed by default)

Usage: The `Providers` component is imported and used in `src/app/layout.tsx` to wrap the application:

```
import Providers from './providers';  
  
export default function RootLayout({ children }: { children: React.ReactNode }) {  
  return (  
    <html>  
      <body>  
        <Providers>  
          {children}  
        </Providers>  
      </body>  
    </html>  
  );  
}
```

Query Keys

File: `src/constants/queryKeys.ts`

Implementation:

```
export const postsKey = ['posts'] as const;  
export const postKey = (id: number) => ['post', id] as const;  
  
export const QueryKeys = {  
  posts: postsKey,  
  post: postKey,  
} as const;
```

Pattern:

- Query keys are defined as **const arrays** for type safety
- Keys follow a hierarchical pattern: `['resource']` for collections, `['resource', id]` for individual items
- Keys are exported through a `QueryKeys` object for consistent usage

Example Usage:

```
// Invalidate all posts queries  
queryClient.invalidateQueries({ queryKey: QueryKeys.posts });  
  
// Invalidate a specific post query  
queryClient.invalidateQueries({ queryKey: QueryKeys.post(123) });
```

Note: While these query keys are defined, they are currently not used in the production news feature (which relies on server-side fetching). They demonstrate the pattern that would be used for future client-side data fetching.

Error Handling Strategy

Files:

- `src/services/httpClient.ts`
- `src/services/requestService.ts`

Error Normalization Function:

```
export function toError(e: any): Error {
  const message = e?.response?.data?.message || e?.message || 'Request failed';
  const err = new Error(message);
  (err as any).status = e?.response?.status;
  (err as any).details = e?.response?.data;
  return err;
}
```

HTTP Request Wrappers:

```
export async function get<T>(url: string, config?: AxiosRequestConfig) {
  try {
    const res = await httpClient.get<T>(url, config);
    return res.data;
  } catch (e) {
    throw toError(e);
  }
}

export async function post<T, B = unknown>(url: string, body?: B, config?: AxiosRequestConfig) {
  try {
    const res = await httpClient.post<T>(url, body, config);
    return res.data;
  } catch (e) {
    throw toError(e);
  }
}
```

Error Object Structure: When a request fails, React Query (or any caller) receives an `Error` object with:

- `message` : Human-readable error message (from API response or default)
- `status` : HTTP status code (if available from axios response)
- `details` : Full response data object (if available)

Benefits:

- Consistent error structure across all API calls
- Type-safe error handling
- Easy access to HTTP status codes for conditional UI rendering
- Full error details preserved for debugging

Data Structures

News API Response

File: `src/interfaces/news.ts`

Type Definitions:

```
export interface NewsSource {
  id?: string;
  name: string;
  url?: string;
  country?: string;
}

export interface NewsArticle {
  title: string;
  description: string;
  content?: string;
  url: string;
  image?: string;
  publishedAt?: string;
  source: NewsSource | string;
}

export interface NewsApiResponse {
  totalArticles?: number;
  articles: NewsArticle[];
  error?: string;
}
```

Field Descriptions:

Field	Type	Required	Description
totalArticles	number	Optional	Total number of articles available (may not match articles.length due to pagination)
articles	NewsArticle[]	Required	Array of news articles
error	string	Optional	Error message (only present when local validation fails)

NewsArticle Fields:

Field	Type	Required	Description
title	string	Required	Article headline
description	string	Required	Article summary/description
content	string	Optional	Full article content (may be truncated)
url	string	Required	Link to original article
image	string	Optional	Article image URL
publishedAt	string	Optional	ISO 8601 timestamp
source	NewsSource string	Required	Source information (object with details or simple string)

Example Response:

```
{
  "totalArticles": 10,
```

```
"articles": [
  {
    "title": "Breaking: Major Technology Breakthrough",
    "description": "Scientists announce revolutionary discovery...",
    "content": "In a groundbreaking announcement today...",
    "url": "https://example.com/article1",
    "image": "https://picsum.photos/400/300?random=1",
    "publishedAt": "2024-01-01T12:00:00Z",
    "source": {
      "name": "Tech News",
      "url": "https://example.com"
    }
  }
]
```

Environment Variables

Required:

- `GNEWS_API_KEY` : API key for GNews v4 API

Optional:

- `USE MOCK NEWS DATA` : Set to `'true'` to use mock data instead of calling GNews API (useful for development/testing)
- `NEXT_PUBLIC_API_BASE_URL` : Base URL for other HTTP clients if needed (not required for the news feature)

Summary

- **Third-Party API:** GNews v4 (`top-headlines` and `search` endpoints)
- **Internal API:** Next.js route handler at `/api/news` that proxies to GNews with fallback to mock data
- **React Query:** Configured with `QueryClientProvider` , query keys pattern, and standardized error handling
- **Data Flow:** Server-side fetching via `fetchNews()` → Next.js API route → components rendering news content
- **Error Resilience:** API failures gracefully fall back to mock data, ensuring the application always provides content to users